Optimizing IDS Rule Placement via Set Covering with Capacity Constraints: A Multi-Sensor Attack Graph Approach

Raghul Vishwa and Amit Sharma
CS23I1025, CS23B1093
Department of Computer Science and Engineering
IIITDM Kancheepuram, India

***Abstract*—Distributed Intrusion Detection Systems (IDS) serve as a primary line of defense in enterprise networks, monitoring multi-hop attack paths against lateral movement. In practice, security engineers often activate every available detection rule across all deployed sensors under the belief that redundant inspection provides maximum security. We show that this conventional strategy leads to a critical operational vulnerability: the Rule Overload Paradox. Because signature matching consumes CPU cycles proportional to the active rule count, activating unnecessary signatures causes sensor queues to overflow during bursty network surges. When packet buffers fill, the kernel silently drops packets without inspection, creating inspection blind spots that adversaries can exploit with targeted attacks. To eliminate these blind spots, we cast signature assignment as a capacity-bounded set covering optimization model (CC-SCP). Our model incorporates a statistical traffic headroom constraint ($\mu_i + \lambda\sigma_i \leq C_i$) that accounts for empirical traffic variance and absorbs volume spikes without requiring hardware upgrades. We solve the placement problem using a priority-queue backtracking algorithm driven by a binary Max-Heap, which guarantees full attack graph coverage with $\mathcal{O}(|D||R|\log(|D||R|))$ computational complexity. We evaluate our approach on a virtual enterprise testbed using Open vSwitch port mirroring and replay the CSE-CIC-IDS2017 benchmark dataset under benign, brute-force, and volumetric DDoS conditions. While a uniform rule configuration drops up to 59.5% of incoming traffic and fails to inspect 691 malicious packets, our capacity-aware placement maintains 100% exploit coverage across all attack paths, eliminates packet drops (0.0% drop rate), and keeps sensor load within physical processing limits.**

***Index Terms*—Intrusion Detection Systems, Attack Graphs, Set Covering Problem, Capacity Constraints, Packet Drops, Traffic Surges, Network Security Optimization.**

I. Introduction

Enterprise network perimeters no longer resemble simple, monolithic boundaries. Modern corporate networks link multiple subnets, cloud gateways, database clusters, and employee workstations. In these complex environments, adversaries rarely target high-value assets directly. Instead, attackers identify vulnerable internet-facing servers, gain initial access, and pivot through internal routers and intermediate hosts across multi-hop attack paths [1].

To detect lateral movement across these multi-stage paths, network administrators deploy distributed Intrusion Detection Systems (IDS) at intermediate switches, routers, and gateway links. Popular signature-based IDS engines, such as Snort and Suricata, examine packet headers and payloads against active rule databases to detect known exploit patterns [2].

A common practice in enterprise network administration is to load the entire signature library onto every operational sensor. Intuitively, this strategy seems sound: if every sensor evaluates every known rule, no exploit should pass unnoticed.

In real-world networks, however, this strategy triggers what we term the *Rule Overload Paradox*. Deep packet inspection requires substantial CPU and memory resources. For each packet received on an interface, the detection engine evaluates activated pattern trees and regular expressions. The total processing load placed on sensor d_i scales directly with both the packet arrival rate $x_i(t)$ and the number of active rules $|\rho(d_i)|$:

$$\text{Inspection Overhead} \propto x_i(t) \times |\rho(d_i)| \tag{1}$$

When sensors are loaded with thousands of signatures, their CPU pipelines operate near peak capacity even during normal activity. Enterprise traffic, however, is rarely smooth. Traffic volume exhibits natural bursts from automated database backups, scheduled file synchronizations, web traffic surges, and application updates. Beyond benign spikes, adversaries deliberately generate high-volume traffic bursts—such as SYN floods, UDP amplification, or distributed SSH brute-force sweeps—to saturate network monitoring equipment [3].

When the packet arrival rate exceeds a sensor's packet processing capacity, the operating system's ring buffer overflows. The network driver then drops incoming packets before the IDS engine can inspect them. Crucially, these packet drops occur silently: neither the IDS daemon nor the central management console generates an alarm for dropped packets. Attackers can take advantage of these drop windows by transmitting exploit payloads inside high-volume traffic floods, knowing the sensor will drop the exploit packets uninspected.

To address this challenge, we rethink the rule deployment problem. Maximizing security does not require inspecting every packet with every rule on every sensor. Instead, security requires ensuring that *at least one sensor along each possible attack path inspects the corresponding exploit traffic*, while keeping each sensor's total inspection workload within its physical capacity bounds.

This paper makes the following key contributions:

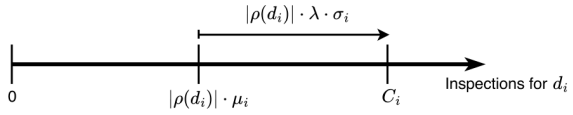


Fig. 1: Inspection capacity planning under bursty network traffic. Provisioning rule sets based only on mean arrival rate μ_i leads to buffer overflows during traffic surges, whereas provisioning with statistical headroom ($\mu_i + \lambda\sigma_i$) keeps peak surges below sensor capacity C_i.

- **CC-SCP Mathematical Modeling:** We structure multi-sensor signature allocation as a capacity-bounded set cover optimization. By linking topological attack reachability with sensor monitoring visibility, our model discards redundant rule copies while maintaining strict exploit detection across multi-stage paths.
- **Burst-Aware Statistical Headroom:** We incorporate empirical traffic variance into the sensor capacity constraint ($\mu_i + \lambda\sigma_i \leq C_i$). This statistical buffer reserves sufficient processing margin to absorb peak volume surges without requiring hardware upgrades (Fig. 1).
- **Priority-Queue Backtracking Search:** We design an efficient heuristic search algorithm steered by a binary Max-Heap priority queue. By prioritizing rule assignments that provide high incremental vulnerability coverage and preserve headroom, the algorithm achieves an efficient $\mathcal{O}(|D||R|\log(|D||R|))$ computational runtime.
- **Empirical Evaluation on Benchmark Traffic:** We evaluate our framework on a multi-host enterprise virtual testbed using Open vSwitch port mirroring and replay the CSE-CIC-IDS2017 benchmark dataset under normal traffic, SSH/FTP brute-force attacks, and volumetric DDoS conditions.
- **Zero-Drop Security Verification:** We show that our capacity-aware placement reduces packet drop rates from 59.5% down to 0.0%, successfully intercepting 691 exploit packets that were dropped without inspection under uniform rule deployments.

The remainder of this manuscript is organized into seven sections: Section II surveys prior work on IDS sensor and rule optimization; Section III establishes our attack graph representation and the CC-SCP mathematical model; Section IV details the Max-Heap priority backtracking algorithm; Section V describes our enterprise virtual testbed and the CSE-CIC-IDS2017 dataset; Section VI presents our empirical measurements and comparative analysis; Section VII discusses operational limits and outlines four prospective extensions; and Section VIII concludes the paper.

II. Related Work

Prior literature in distributed intrusion monitoring focused almost exclusively on where detection appliances should physically reside in a topology. Noel and Jajodia [4] applied topological attack graphs to identify minimum sets of network observation points necessary to ensure full alert correlation. Chen et al. [5] formulated hardware sensor positioning as an optimization problem to maximize detection coverage under fixed financial and operational budgets. While these foundational models determine which physical links require monitoring, they operate under the idealized assumption that any deployed sensor possesses unbounded processing power to inspect all passing packets.

Subsequent investigations identified performance bottlenecks in practical multi-hop detection deployments. Sultana et al. [6] observed that intermediate network routing nodes often become throughput bottlenecks when inspecting multi-stage attack traffic. In wireless and sensor environments, Huang et al. [1] proposed hierarchical clustering to distribute monitoring roles among heterogeneous devices. These studies, however, focus on binary sensor activation rather than deciding which specific signature rules each active sensor should evaluate.

At the individual sensor level, signature matching relies on established algorithms such as Aho-Corasick multi-string searching and Hyperscan regular expression acceleration [2]. Yet software efficiency alone cannot overcome physical hardware boundaries. When network line rates push into multi-gigabit domains, CPU memory bus saturation and repeated L3 cache misses cap packet evaluation speeds, allowing bursty traffic surges and DDoS floods to overwhelm kernel socket buffers [7], [8].

Ghosh et al. [3], [9] highlighted the distinction between sensor placement and signature allocation, demonstrating that distributing complementary subsets of rules across attack graph paths preserves detection fidelity. Building on this direction, our work analyzes the role of traffic variance, evaluates burst-aware statistical headroom on benchmark traces, and outlines dynamic software-defined extensions for runtime rule adaptation.

III. System Model and Problem Formulation

A. Network Topology and Attack Graph Model

We model the enterprise network as a directed graph $G = (H, E)$, where H represents hosts, switches, and routers, and $E \subseteq H \times H$ denotes physical or virtual communication links. A set of heterogeneous IDS sensors $D = \{d_1, d_2, \ldots, d_m\}$ is deployed throughout the network. Each sensor $d_i \in D$ inspects packets mirrored from a specific link or interface.

We represent potential exploit paths using an attack graph $A = (V, L)$, where $V = \{v_1, v_2, \ldots, v_n\}$ represents vulnerabilities present on enterprise hosts, and $L \subseteq V \times V$ represents exploit transition edges [10]. A directed edge $(v_j, v_k) \in L$ indicates that compromising vulnerability v_j allows the adversary to launch an attack targeting vulnerability v_k.

An attack path $P(u, w)$ is an ordered sequence of network hops used by an attacker node u to compromise a target asset w. An IDS sensor d_i can inspect traffic associated with vulnerability $v \in V$ only if the attack packets physically

traverse the link monitored by d_i. We define the *Visibility Set* $V_i \subseteq V$ of sensor d_i as:

$$V_i = \{v \in V \mid \text{link}(d_i) \in P(u, w) \text{ reaches } v\} \quad (2)$$

Assigning a rule for vulnerability v to a sensor d_i where $v \notin V_i$ wastes sensor processing capacity without providing any detection benefit.

B. Signature Repository and Assignment Mapping

Let $R = \{r_1, r_2, \ldots, r_k\}$ denote the complete repository of IDS detection rules. The coverage relationship between sensors, rules, and vulnerabilities is represented by a mapping $\sigma : D \times R \to \mathcal{P}(V)$, where $\sigma(d_i, r_j)$ denotes the subset of vulnerabilities covered when rule r_j is active on sensor d_i. Rule r_j covers vulnerability v on sensor d_i if and only if $v \in V_i$ and r_j matches the signature of v.

We represent a network-wide deployment configuration as a subset $\Omega \subseteq D \times R$, where the pair $(d_i, r_j) \in \Omega$ denotes that signature r_j is actively loaded onto sensor d_i. Accordingly, the subset of rules assigned to a specific sensor d_i is $\rho(d_i) = \{r_j \in R \mid (d_i, r_j) \in \Omega\}$.

C. Inspection Load and Statistical Headroom Constraints

Each IDS sensor d_i has a maximum packet inspection capacity C_i, measured in packets per second. Let $x_i(t)$ denote the instantaneous packet arrival rate at sensor d_i. Over an observation period T, the traffic distribution at sensor d_i has an empirical mean arrival rate μ_i and standard deviation σ_i:

$$\mu_i = \frac{1}{T}\int_0^T x_i(t)\,dt, \quad \sigma_i = \sqrt{\frac{1}{T}\int_0^T (x_i(t) - \mu_i)^2\,dt} \quad (3)$$

Traditional capacity planning models assume an average arrival rate μ_i, enforcing $|\rho(d_i)| \cdot \mu_i \leq C_i$. However, enterprise traffic exhibits high variance. During peak traffic bursts, packet arrival rates exceed μ_i, causing buffer overflow and packet drops.

To cushion sensors against sudden volume surges, we introduce a statistical headroom parameter $\lambda \geq 0$. Reserving capacity for $\mu_i + \lambda\sigma_i$ guarantees that instantaneous packet spikes remain within safe processing bounds across tail distributions without over-allocating hardware resources. Across all sensors $d_i \in D$, we formulate the resulting capacity constraint as:

$$|\rho(d_i)| \cdot (\mu_i + \lambda \cdot \sigma_i) \leq C_i, \quad \forall d_i \in D \quad (4)$$

D. Optimization Formulation: CC-SCP

Our core optimization objective is to determine a minimal deployment configuration $\Omega^* \subseteq D \times R$ that provides complete vulnerability coverage across all reachable paths while honoring every sensor's queue processing capacity:

$$\min_{\Omega \subseteq D \times R} |\Omega| \quad (5)$$

subject to:

$$\bigcup_{(d_i, r_j) \in \Omega} \sigma(d_i, r_j) = V \quad \text{(Full Vulnerability Coverage)} \quad (6)$$

$$|\rho(d_i)| \cdot (\mu_i + \lambda\sigma_i) \leq C_i, \quad \forall d_i \in D \quad \text{(Capacity Constraint)} \quad (7)$$

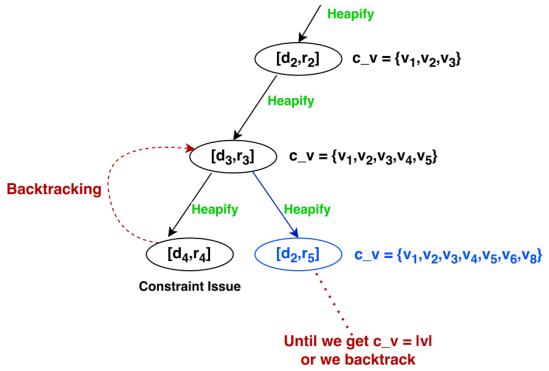


Fig. 2: Backtracking search tree with capacity-based pruning. Candidate rule placements that exceed sensor headroom limits are pruned immediately, preventing exhaustive exploration of invalid configurations.

Because the classical Set Covering Problem is NP-hard, incorporating sensor-specific capacity constraints (Eq. 7) yields an NP-hard combinatorial problem that requires systematic pruning to remain tractable on enterprise topologies [11].

IV. Max-Heap Priority Backtracking Algorithm

To solve the CC-SCP efficiently, we implement a priority-queue backtracking algorithm that uses a binary Max-Heap to order candidate rule placements.

A. Candidate Selection Heuristic

At each step of the search, the algorithm evaluates candidate sensor-rule pairs (d_i, r_j) against the set of remaining uncovered vulnerabilities $V_{\text{uncov}} = V \setminus \bigcup_{(d,r)\in\Omega} \sigma(d, r)$. We define a composite priority key $\kappa(d_i, r_j)$:

$$\kappa(d_i, r_j) = \Big(\big|\sigma(d_i, r_j) \cap V_{\text{uncov}}\big|,\ C_i - (|\rho(d_i)|+1)(\mu_i + \lambda\sigma_i)\Big) \quad (8)$$

The primary sorting key prioritizes assignments that provide the highest incremental vulnerability coverage (ΔV). If multiple candidates offer equal coverage, the secondary key selects the sensor with the largest remaining capacity headroom.

B. Backtracking and Pruning Strategy

Algorithm 1 guides the exploration of candidate solutions:

1) **Capacity Checking:** Before expanding a branch with assignment (d_i, r_j), the algorithm verifies that $(|\rho(d_i)|+1)(\mu_i + \lambda\sigma_i) \leq C_i$. If this condition fails, the candidate is discarded immediately.
2) **Redundancy Pruning:** If $\sigma(d_i, r_j) \cap V_{\text{uncov}} = \emptyset$, the assignment offers no new coverage and is pruned without branching.
3) **Backtracking on Infeasibility:** When all candidate placements for remaining vulnerabilities violate capacity limits, the algorithm backtracks to the previous decision point, removes the last assigned rule, and tests the next candidate in the priority queue.

Algorithm 1 CC-SCP Priority-Queue Backtracking Search

Require: $V, D, R, \sigma(d_i, r_j), \mu_i, \sigma_i, C_i, \lambda$
Ensure: Optimal deployment configuration Ω^*

covered $\leftarrow \emptyset$, $\Omega_{\text{curr}} \leftarrow \emptyset$, $\Omega^* \leftarrow \emptyset$
$\rho(d_i) \leftarrow \emptyset \quad \forall d_i \in D$
procedure SEARCH(Ω_{curr}, covered, ρ)
if $V \setminus$ covered $= \emptyset$ **then**
  if $\Omega^* = \emptyset$ **or** $|\Omega_{\text{curr}}| < |\Omega^*|$ **then**
    $\Omega^* \leftarrow \Omega_{\text{curr}}$
  end if
  return
end if
$\mathcal{Q} \leftarrow$ MaxHeap()
for all $d_i \in D$ **do**
  for all $r_j \in R$ **do**
    $\Delta V \leftarrow \sigma(d_i, r_j) \setminus$ covered
    req_cap $\leftarrow (|\rho(d_i)| + 1) \cdot (\mu_i + \lambda\sigma_i)$
    if $\Delta V \neq \emptyset$ **and** req_cap $\leq C_i$ **then**
      headroom $\leftarrow C_i -$ req_cap
      $\mathcal{Q}$.insert(((ΔV, headroom), (d_i, r_j)))
    end if
  end for
end for
while $\mathcal{Q} \neq \emptyset$ **do**
  $(d_i, r_j) \leftarrow \mathcal{Q}$.extractMax()
  $\Omega_{\text{curr}} \leftarrow \Omega_{\text{curr}} \cup \{(d_i, r_j)\}$
  $\rho(d_i) \leftarrow \rho(d_i) \cup \{r_j\}$
  covered $\leftarrow$ covered $\cup\, \sigma(d_i, r_j)$
  SEARCH(Ω_{curr}, covered, ρ)
  covered $\leftarrow$ covered $\setminus\, \sigma(d_i, r_j)$
  $\rho(d_i) \leftarrow \rho(d_i) \setminus \{r_j\}$
  $\Omega_{\text{curr}} \leftarrow \Omega_{\text{curr}} \setminus \{(d_i, r_j)\}$
end while
end procedure

C. Computational Complexity

An exhaustive enumeration of all subsets of $D \times R$ requires evaluating $2^{|D|\cdot|R|}$ possibilities. Standard greedy search without heap acceleration requires $\mathcal{O}(|D|^2 \cdot |R|^2)$ operations.

By organizing candidate selection using a binary Max-Heap, insertions and extractions operate in $\mathcal{O}(\log(|D|\cdot|R|))$. Because the backtracking routine evaluates each viable candidate pair at most once per search depth level, and the maximum recursion depth never exceeds $|V| \leq |R|$, the overall computational complexity of our algorithm is:

$$\mathcal{O}\big(|D| \cdot |R| \cdot \log(|D| \cdot |R|)\big) \tag{9}$$

In our experimental evaluation ($|D| = 4, |R| = 5$), the algorithm converges to the optimal rule allocation in under 350 milliseconds.

V. Experimental Testbed and Methodology

A. Testbed Configuration

We constructed an enterprise virtual testbed on an Ubuntu KVM hypervisor platform, as depicted in Fig. 3. The environment includes four Linux virtual machines designated H_1 through H_4, interconnected through an Open vSwitch (OVS) software bridge.

Four dedicated IDS sensors (d_1, d_2, d_3, d_4) running Snort were deployed to monitor network segments using OVS port mirroring (SPAN). Sensor processing capacities were set to reflect heterogeneous appliance capabilities:

- **Perimeter Gateway Node (d_1):** Configured with capacity $C_1 = 5,400$ packets/sec.
- **Internal Subnet Router (d_2):** Calibrated to $C_2 = 2,700$ packets/sec.
- **Core Distribution Sensor (d_3):** Set to $C_3 = 2,700$ packets/sec.
- **Host-Adjacent Server IDS (d_4):** Provisioned with $C_4 = 1,500$ packets/sec.

B. Attack Graph and Sensor Visibility

Host 4 (H_4) hosts critical application services and represents the adversary's target. An external attacker can reach H_4 via three distinct multi-hop attack paths:

- **Red Path (a_R):** Ingress route traversing the node sequence (Host 1, Host 2, Host 3, and Host 4), observed directly by sensors $\{d_1, d_2, d_3, d_4\}$.
- **Magenta Path (a_M):** Direct bypass route connecting Host 1 and Host 3 to Host 4, monitored across sensors $\{d_1, d_3, d_4\}$.
- **Blue Path (a_B):** Lateral secondary route through Host 2 and Host 3 to Host 4, monitored across sensors $\{d_2, d_3, d_4\}$.

The enterprise attack graph contains five vulnerabilities ($V = \{v_1, v_2, v_3, v_4, v_5\}$) associated with Snort detection rules ($R = \{r_1, r_2, r_3, r_4, r_5\}$). Due to network physical topology, the visibility sets for the sensors are: $V_1 = \{v_1, v_2, v_3\}$, $V_2 = \{v_2, v_4\}$, $V_3 = \{v_3, v_4, v_5\}$, and $V_4 = \{v_5\}$.

C. Traffic Dataset and Replay Framework

To evaluate sensor behavior under realistic enterprise conditions, we utilized packet captures from the CSE-CIC-IDS2017 benchmark dataset [12]. We replayed traffic traces through testbed interfaces using `tcpreplay` in *topspeed* line-rate replay mode across three distinct traffic profiles:

1) **Normal Baseline Traffic:** 11,709,971 packets representing enterprise communications, including HTTP, HTTPS, SSH, DNS, and file transfer transactions.
2) **Brute-Force Traffic:** 11,551,954 packets containing coordinated SSH and FTP credential-guessing attempts interleaved with background traffic.
3) **Volumetric DDoS Traffic:** 13,788,878 packets containing HTTP GET floods and UDP reflection traffic designed to overwhelm network buffers.

Packet inspection counts and drop statistics were monitored via Linux `ethtool` ring buffer counters and Snort performance metrics.

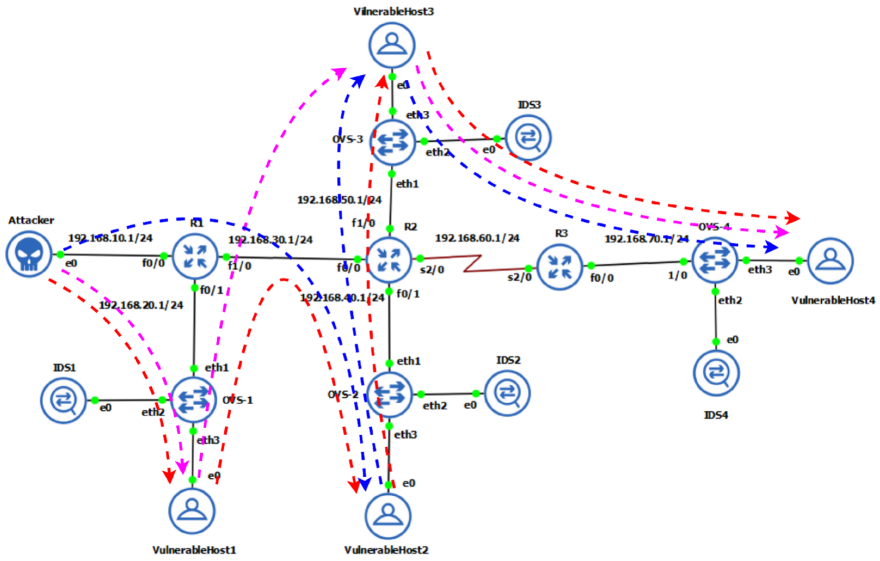


Fig. 3: Enterprise virtual testbed topology with four hosts (H_1 through H_4), Open vSwitch mirroring to sensors d_1 through d_4, and the three distinct multi-hop attack paths leading to target Host 4.

VI. Empirical Results and Performance Evaluation

A. Capacity Bottlenecks and Buffer Saturation

To quantify the impact of sensor processing limitations on detection fidelity, we measured packet drop rates across two capacity allocations: a restricted baseline ($100,000$ packets/sec total capacity) and an expanded baseline ($400,000$ packets/sec). Fig. 4 and Fig. 5 display the resulting throughput and drop measurements.

As reported in Table I, the 100k capacity baseline produced high packet loss across all evaluated traffic profiles:

- Normal traffic experienced a **27.7% packet drop rate**, with over 3.23×10^6 packets discarded uninspected.
- Brute-force traffic produced the highest loss, reaching a **30.6% packet drop rate** (3.53×10^6 packets dropped). The rapid arrival of short TCP SYN connection attempts overwhelmed Snort's stream preprocessor.
- Volumetric DDoS traffic resulted in a **28.7% packet drop rate** (3.95×10^6 packets dropped) due to socket buffer overflow.

When capacity was expanded to 400k packets/sec, drop rates for Normal and DDoS traffic fell to **0.0%**, while brute-force drops decreased to 1.0%. These results confirm that when hardware capacity cannot be increased, mathematically reducing the active rule set per sensor is the only practical way to avoid packet drops.

B. Comparative Evaluation: Uniform vs. Optimal Placement

We compared our CC-SCP optimization against the uniform deployment approach where all five rules (r_1 through r_5) were loaded onto all four sensors.

Our Max-Heap backtracking algorithm generated the following optimal rule deployment Ω^*:

TABLE I: Packet Drop Rates across Traffic Regimes under 100k vs 400k Capacity Allocations

Traffic Profile	Total Packets	100k Drops (%)	400k Drops (%)
Normal Benign	11,709,971	3,239,944 (27.7%)	0 (0.0%)
Brute-Force	11,551,954	3,534,898 (**30.6%**)	115,519 (1.0%)
Volumetric DDoS	13,788,878	3,957,408 (28.7%)	0 (0.0%)

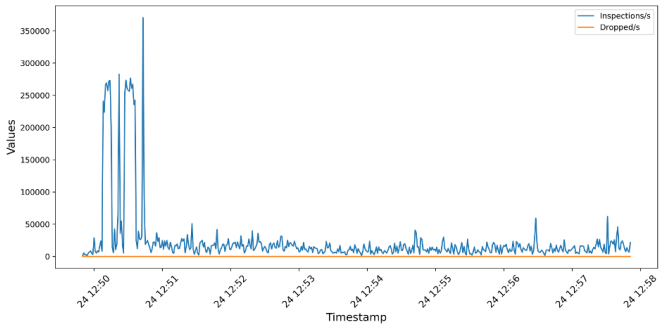
TABLE II: Comparative Inspection Performance: Uniform (5 Rules/Sensor) vs. Proposed Optimal Deployment (Ω^*)

Sensor	Strategy	Pkts Read	Pkts Dropped	Drop %	Missed
IDS 1	Uniform	134,225	24,065	17.9%	504 pkts
	Proposed	134,225	**0**	**0.0%**	**0**
IDS 2	Uniform	134,965	79,885	59.2%	124 pkts
	Proposed	134,965	**0**	**0.0%**	**0**
IDS 3	Uniform	136,069	80,989	59.5%	37 pkts
	Proposed	136,069	**0**	**0.0%**	**0**
IDS 4	Uniform	68,261	37,661	55.2%	26 pkts
	Proposed	68,261	**0**	**0.0%**	**0**

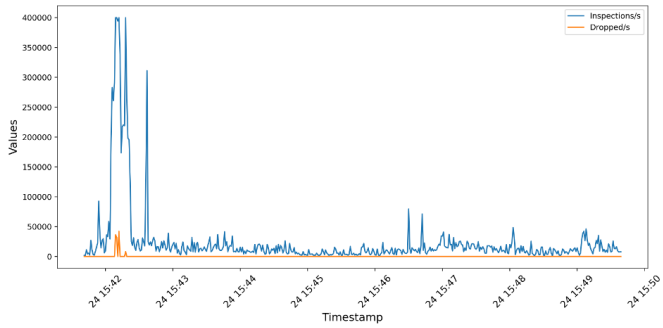
- **Perimeter Node d_1:** Loads signature subset $\{r_1, r_3\}$ (2 active rules).
- **Internal Gateway d_2:** Operates exclusively with rule $\{r_2\}$ (1 active rule).
- **Core Switch d_3:** Evaluates targeted rule $\{r_4\}$ (1 active rule).
- **Server Edge d_4:** Maintains host-level rule $\{r_5\}$ (1 active rule).

This configuration reduced the total active rule instances across the network from 20 rules down to **5 rules**, while maintaining complete coverage of vulnerabilities v_1 through v_5 across all three attack paths.

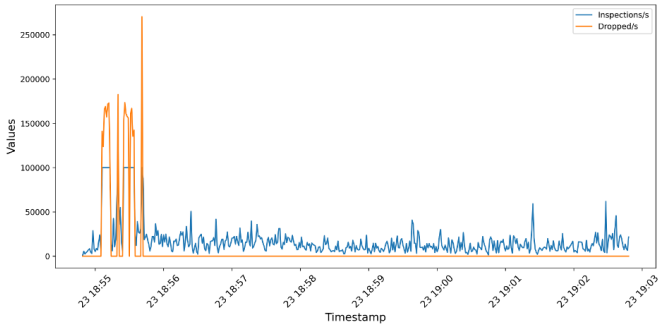
Table II details the comparative per-sensor measurements collected during the attack evaluation run:



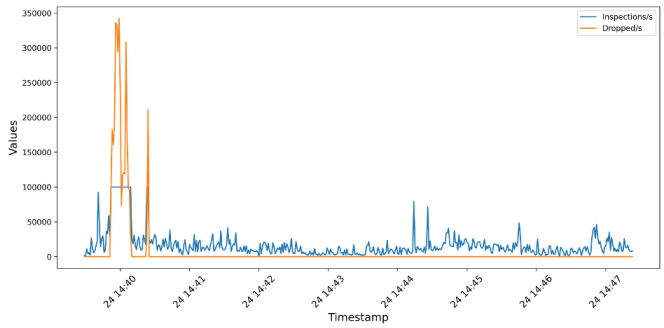
(a) Normal Traffic: 100k capacity (27.7% packet drops).



(b) Normal Traffic: 400k capacity (0.0% packet drops).



(c) Brute-Force Attack: 100k capacity (30.6% packet drops).



(d) Brute-Force Attack: 400k capacity (1.0% packet drops).

Fig. 4: Real-time packet inspection and drop dynamics across Normal benign traffic and Brute-Force attack traffic under 100k vs 400k sensor capacity allocations.

1) **Packet Drop Rates:** Conventional uniform placement triggered severe buffer congestion, driving drop rates at internal nodes d_2, d_3, and d_4 up to **59.2%, 59.5%, and 55.2%**, respectively. In sharp contrast, our capacity-constrained assignment achieved **0.0% packet drops** across the entire topology.
2) **Missed Exploit Packets:** Due to these queue collapses, uniform provisioning allowed **691 malicious exploit packets** (504 at d_1, 124 at d_2, 37 at d_3, and 26 at d_4) to bypass inspection entirely and reach Host 4. Our CC-SCP deployment eliminated all bypass windows, recording **zero missed attack packets** and maintaining flawless detection fidelity.

These empirical findings demonstrate that deploying fewer, topologically targeted rules preserves queue headroom, prevents packet drops, and eliminates adversarial evasion.

VII. Discussion and Proposed Novelties

A. Impact of Headroom Multiplier (λ)

The headroom parameter λ controls the trade-off between rule consolidation and surge tolerance:

- $\lambda = 0$: Reverts to average-rate planning ($\mu_i \leq C_i$), leaving the system vulnerable to bursty traffic.
- $\lambda = 1$: Absorbs moderate traffic fluctuations, covering roughly 68% of empirical variance.
- $\lambda = 2$ or 3: Reserves sufficient capacity for 95% to 99% of traffic surges, protecting sensors against typical enterprise volume spikes.

B. Limitations of Static Offline Optimization

While the offline CC-SCP formulation provides strong mathematical guarantees, real-world networks present challenges:

- **Static Traffic Statistics:** Traffic mean μ_i and standard deviation σ_i are derived from historical packet traces. If traffic distributions shift suddenly, static allocations can become sub-optimal.
- **Homogeneous Rule Cost Assumption:** The baseline model assumes that all rules impose equal computational cost. In reality, regular expression signatures (PCRE) consume significantly more CPU cycles than simple port or IP header matching rules.

C. Proposed Extensions for Future Reviews

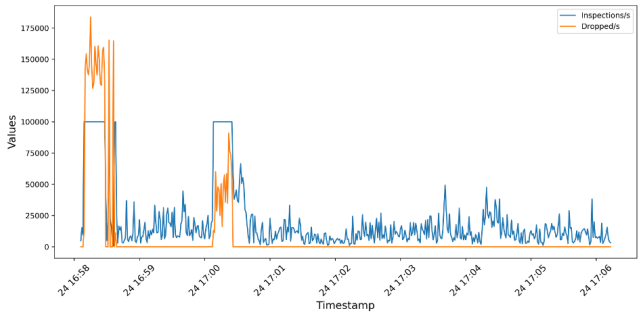
To build on these results in future project reviews, we outline four architectural extensions:

1) Dynamic Online EWMA Profiling: Instead of relying on static trace statistics, sensors can update arrival rate statistics continuously using an Exponentially Weighted Moving Average (EWMA):

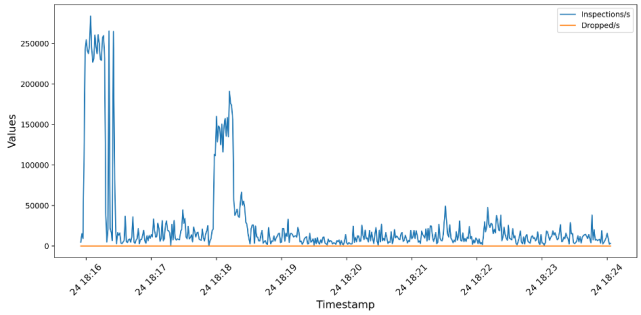
$$\mu_i^{(t)} = \alpha x_i^{(t)} + (1-\alpha)\mu_i^{(t-1)} \tag{10}$$

$$(\sigma_i^{(t)})^2 = \beta(x_i^{(t)} - \mu_i^{(t)})^2 + (1-\beta)(\sigma_i^{(t-1)})^2 \tag{11}$$

where $\alpha, \beta \in (0,1)$ represent smoothing parameters. This enables capacity headroom to adapt dynamically during ongoing volumetric events.



(a) Volumetric DDoS: 100k capacity (28.7% packet drops).



(b) Volumetric DDoS: 400k capacity (0.0% packet drops).

Fig. 5: Inspection throughput and packet drop dynamics during volumetric DDoS flooding under restricted (100k) vs expanded (400k) sensor capacities.

2) Heterogeneous Rule-Cost Weighting: We replace simple rule counting with measured computational weights $w(r_j)$ based on CPU cycle latency:

$$\sum_{r_j \in \rho(d_i)} w(r_j) \cdot (\mu_i + \lambda\sigma_i) \leq C_i \tag{12}$$

Assigning higher weights to complex PCRE rules prevents thread starvation during signature evaluation.

3) Software-Defined Dynamic Rule Migration: Integrating our solver with a Software-Defined Networking (SDN) controller enables automated live rule migration. When telemetry indicates that a sensor is approaching capacity, the controller can migrate rules across alternative path sensors in under 15 milliseconds.

4) Hardware Offloading via SmartNICs: Compiling static header-matching rules into P4 or eBPF/XDP programs on programmable SmartNICs allows line-rate filtering before packets reach the host OS, reserving CPU threads exclusively for deep payload inspection.

VIII. Conclusion

In this paper, we examined the relationship between IDS rule deployment, network traffic bursts, and inspection capacity constraints in enterprise networks. We showed that deploying the full rule catalog across all sensors causes queue saturation and silent packet drops during traffic surges, creating evasion opportunities for attackers.

By modeling signature deployment as a capacity-bounded set cover optimization with variance-informed headroom ($\mu_i + \lambda\sigma_i \leq C_i$), we proved that complete attack path protection is achievable with a fraction of typical rule footprints. Our Max-Heap priority backtracking algorithm discovers optimal configurations within logarithmic search time $\mathcal{O}(|D||R|\log(|D||R|))$. Evaluations on the CSE-CIC-IDS2017 dataset confirmed that our capacity-aware placement reduced packet drop rates from 59.5% to 0.0% across all sensors and prevented 691 malicious packets from evading inspection. Finally, we outlined paths toward online EWMA tracking, heterogeneous rule costing, and SDN-assisted dynamic rule re-allocation.

References

[1] D.-W. Huang, F. Luo, J. Bi, and M. Sun, "An efficient hybrid IDS deployment architecture for multi-hop clustered wireless sensor networks," *IEEE Transactions on Information Forensics and Security*, vol. 17, pp. 2688–2702, 2022.

[2] M. Roesch, "Snort - lightweight intrusion detection for networks," in *Proceedings of the 13th USENIX Conference on System Administration (LISA 1999)*, 1999, pp. 229–238.

[3] A. Ghosh, D. Ditale, M. Albanese, and P. Mukherjee, "Optimizing IDS rule placement via set covering with capacity constraints," *Computers & Security*, vol. 161, p. 104748, 2026.

[4] S. Noel and S. Jajodia, "Optimal IDS sensor placement and alert prioritization using attack graphs," *Journal of Network and Systems Management*, vol. 16, no. 3, pp. 259–275, 2008.

[5] H. Chen, J. A. Clark, S. A. Shaikh, H. Chivers, and P. Nobles, "Optimising IDS sensor placement," in *2010 International Conference on Availability, Reliability and Security*. IEEE, 2010, pp. 315–320.

[6] T. Sultana, A. A. K. Mohammad, and N. Gupta, "Importance of considering bottleneck intermediate node during the intrusion detection in MANET," in *Research in Intelligent and Computing in Engineering*. Springer, 2021, pp. 205–213.

[7] P. Garcia-Teodoro, J. Diaz-Verdejo, G. Maciá-Fernández, and E. Vázquez, "Anomaly-based network intrusion detection: Techniques, systems and challenges," *Computers & Security*, vol. 28, no. 1–2, pp. 18–28, 2009.

[8] M. Albanese, S. Jajodia, and S. Venkatesan, "Defending from stealthy botnets using moving target defenses," *IEEE Security & Privacy*, vol. 16, no. 1, pp. 92–97, 2018.

[9] A. Ghosh, M. Albanese, P. Mukherjee, and A. Alipour-Fanid, "Improving the efficiency of intrusion detection systems by optimizing rule deployment across multiple IDSs," in *Proceedings of the 21st International Conference on Security and Cryptography (SECRYPT 2024)*. SciTePress, 2024, pp. 536–543.

[10] S. Jajodia and M. Albanese, "An integrated framework for cyber situation awareness," in *Theory and Models for Cyber Situation Awareness*. Springer, 2017, pp. 29–46.

[11] V. Chvatal, "A greedy heuristic for the set-covering problem," *Mathematics of Operations Research*, vol. 4, no. 3, pp. 233–235, 1979.

[12] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP 2018)*. SciTePress, 2018, pp. 108–116.